Calyr_ai_syntax

Concept and Architecture
small reduced presentation edition

CALYRAI × BOKU VIENNA

Institute of Biophysics / University of Natural Resources and Life Sciences, Vienna

symbolic systems / execution graphs / computational language layer

Problem

- Scientific code tends to fracture across notebooks, scripts, and engine-specific wrappers.
- The semantic object is usually lost once execution starts.
- Calyr needs a syntax layer that is symbolic first and executable second.

Design stance

Reduced surface.
Explicit structure.
Engine-agnostic semantics.

Core Idea

Definition

Calyr_ai_syntax is a Mathematica-inspired symbolic expression layer for Python-centric computational workflows.

EX-PRESS

Trees, rules, symbolic forms, explicit operators.

MAP

Bind abstract forms to engines, datasets, and pipelines.

RUN

Preserve structure through execution, logging, and output manifests.

Two Sample Regimes

Downloaded cohort

A broad sample set built from public protein SAXS entries, each carrying measured $I(q)$, optional $P(r)$, metadata, and model attachments.

- many proteins
- one measurement-centric structure
- ideal for representation learning and PCA

Structured sample

A deeper sample object such as SBPA, where AlphaFold, cryo-EM, LAMMPS, and SAXS belong to one coordinated system state.

- one system, many modalities
- one object identity across stages
- ideal for integrative evaluation

Downloaded Protein Cohort

Cohort example

Public SASBDB protein entries such as SASDXF6, SASDYY5, and SASDC44 provide experimental $I(q)$, optional $P(r)$, metadata, and model files in one reproducible layout.

- Each entry is a valid scientific sample object.
- The first stable representation is the measured scattering curve $I(q)$.
- Optional model and $P(r)$ files remain attached as context, not mandatory preprocessing.

Sample Structure: SBPA

Reference sample object

SBPA is the structured reference case: one protein system tied simultaneously to AlphaFold structure, cryo-EM constraints, LAMMPS simulation, and SAXS observables.

- The sample is not a single file. It is a coupled state across modalities.
- AlphaFold provides the starting local structure.
- Cryo-EM fixes the global arrangement and lattice consistency.
- LAMMPS evolves the periodic unit-cell state.
- SAXS provides observable-space evaluation and back-calculation targets.

Example Protein Set

SASDXF6

Measured: scattering.dat
Context: metadata, models
Role: aligned $I(q)$ sample

SASDYY5

Measured: scattering.dat
Optional: pr.dat
Role: curve comparison + real-space check

SASDC44

Measured: scattering.dat
Context: metadata, model links
Role: shared evaluation-space entry

Every downloaded protein enters the same semantic pipeline: measured curve first, optional attachments second, evaluation layer unchanged.

SBPA Modalities

AlphaFold

Input: sequence, confidence, PAE
Output: initial monomer geometry
Role: structural prior

Cryo-EM + LAMMPS

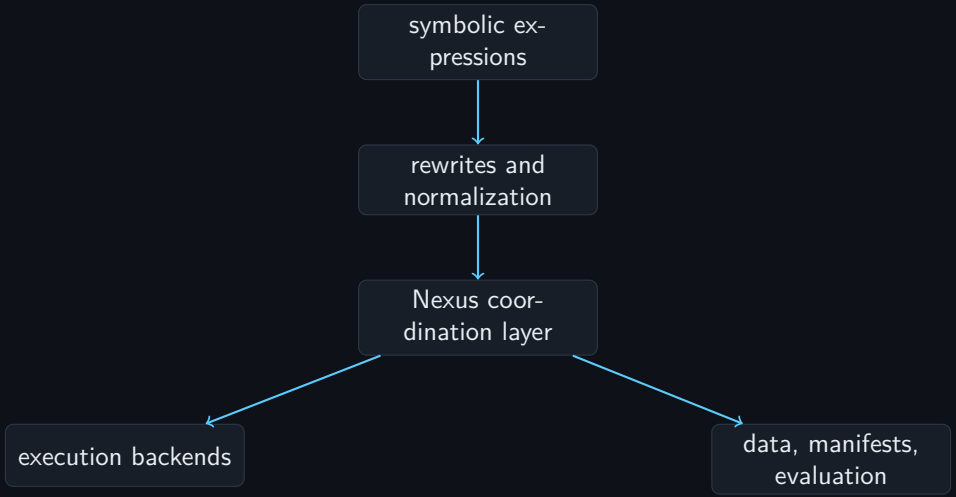
Input: map, fit restraints, unit cell
Output: constrained MD state
Role: dynamic physical refinement

SAXS

Input: measured or back-calculated curves
Output: $I(q)$, optional $P(r)$, PCA features
Role: observable-space evaluation

SBPA is the template for sample structuring: multiple modalities, one object identity, one evaluation layer.

Architecture



Nexus Concept

Definition

Nexus is the operational membrane of Calyr: it connects symbolic intent, engine execution, and data evaluation without changing the meaning level of the scientific object.

- It does not introduce a second language for runtime control.
- It binds one object graph to environments, engines, inputs, and outputs.
- It records each transition as a manifest, so execution remains inspectable and reproducible.

Nexus as Declarative Control Plane

Nexus is not a logbook. Nexus is the control plane of scientific execution.

The system follows a strict declarative paradigm:

$$\text{State}_t \xrightarrow{\text{Nexus Declaration}} \text{Execution} \xrightarrow{\text{Results}} \text{State}_{t+1} \tag{1}$$

Every scientific action is defined as a transition between two fully specified Nexus states.

Correct Execution Paradigm

Incorrect (Imperative)

```
# manual workflow (forbidden)
edit lammps_input.script
run lammps
log results later
```

Correct (Declarative Nexus Flow)

```
# 1. modify Nexus declaration
git commit -m "pipeline:lammps:temp=310"

# 2. execute via Nexus
python -m nexus_engine lammps_experiment

# 3. results are automatically tracked
```

Rule: Nothing exists unless it is declared in Nexus.

Hierarchical Control Structure

$$\text{Nexus}_{root} \rightarrow \text{Nexus}_{domain} \rightarrow \text{Nexus}_{tool} \rightarrow \text{Execution} \tag{2}$$

This defines a multi-level control system:

- **Level 0: Root Nexus** — global policies, FAIR enforcement, scoring, publication gating
- **Level 1: Domain Nexus** — scientific semantics and normalization
- **Level 2: Tool Nexus** — execution bindings and environment definitions
- **Level 3: Experiment Layer** — declared scientific studies and comparisons

State Machine Interpretation

Nexus defines a state machine over scientific reality.

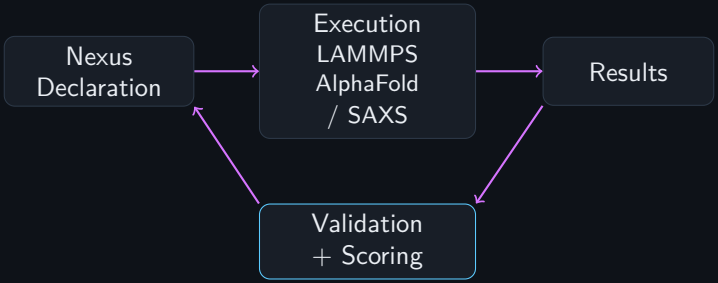
Each Git commit represents a discrete state:

$$S_t = \{\text{parameters}, \text{models}, \text{data}, \text{assumptions}\} \tag{3}$$

$$S_t \rightarrow S_{t+1} \tag{4}$$

The transition function is implemented by Nexus-controlled execution.

Closed-Loop Scientific System



Every result feeds back through validation before Nexus advances to the next declared state.

Separation of Concerns

$$\text{Nexus} = \text{Declarative Layer} \tag{5}$$

$$\text{Tools} = \text{Execution Layer} \tag{6}$$

This separation ensures:

- No hidden parameters
- Full reproducibility
- FAIR compliance by construction
- Cross-domain comparability

Formal Rule Set and Workflow

Formal Rule Set

1. No execution outside Nexus
2. No result without FAIR metadata
3. No experiment without declaration
4. No publication without validation

Operational Workflow

```
# Modify declaration
git commit -m "pipeline:lammps:update-parameters"

# Execute
python -m nexus_engine experiment_name

# Store + track results
git commit -m "data:experiment_name:results"
```

Final Statement

You do not run simulations.
You materialize Nexus states.

PCA Intro for Both Regimes

Input to PCA

A collection of samples is mapped to a common numerical representation, for example downloaded SAXS curves on a fixed q-grid or aligned feature vectors from SBPA simulation and analysis.

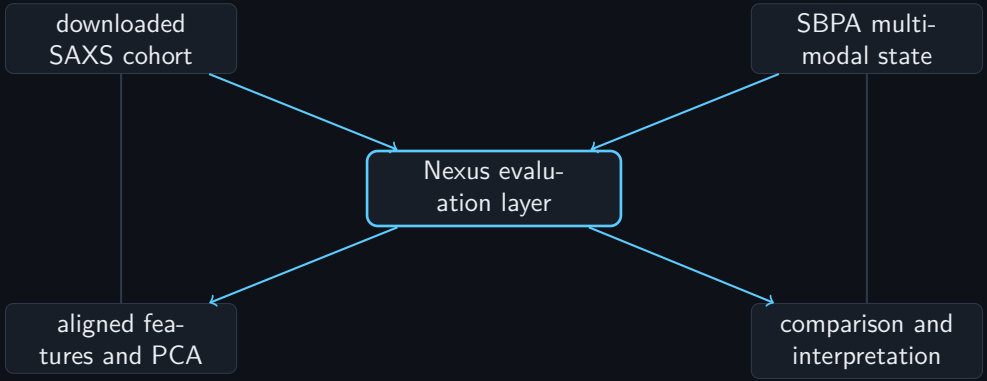
- one row per protein entry, state, frame, or condition
- one aligned feature vector per row
- PCA reveals dominant structural or observable variation modes

Interpretation

The principal components are not new raw data. They are a compact coordinate system for comparing measured cohort samples and structured SBPA states without breaking sample identity.

In the repo workflow, PCA is only meaningful once observables or fitted parameter sets are dimensionally aligned.

Representation Flow



Nexus holds both broad downloaded cohorts and deeply structured multimodal samples, so new setups can enter the same evaluation plane without redesigning the language each time.

One Evaluation Level

Without Nexus

Model, script, raw output, and analysis drift into separate layers with different naming and different state.

- hard to compare runs
- analysis detached from provenance
- semantics collapse into files

With Nexus

The same semantic object is carried from setup to result, so data evaluation stays on one coordinated plane.

- one identity for run and result
- one manifest chain for provenance
- one level for simulation and analysis

Why We Do Not Need Dmax

Key point

Dmax belongs to the optional real-space $P(r)$ description. It is not required to place downloaded SAXS curves or SBPA SAXS observables into a Nexus representation or to run PCA across aligned samples.

- PCA can operate directly on standardized $I(q)$ vectors.
- PCA can also operate on fitted parameter vectors when they have common length.
- Dmax is useful for $P(r)$ interpretation and sanity checks, but it is not a gatekeeper for representation learning.
- This keeps the evaluation layer closer to the measured data and avoids introducing an unnecessary extra assumption early in the pipeline.

Why It Fits Calyr

- **core/** supplies the model vocabulary.
- **engines/** provides realizations such as LAMMPS or SAXS execution.
- **experiments/** defines concrete scientific programs such as SBPA.
- **data/** and **results/** become typed endpoints rather than loose folders.
- **nexus/** keeps model state, execution state, and evaluation state aligned instead of splitting them across ad hoc tools.
- The same framework can hold downloaded SAXS proteins, their embeddings, their fitted parameters, and their downstream comparisons on one semantic level.

Minimal Object Model

Symbolic form

$$\mathrm{Expr}(\mathrm{Head}, \mathrm{Args}, \mathrm{Meta})$$

- Head defines semantic type.
- Args carries ordered structure.
- Meta holds provenance, bindings, constraints, and evaluation context.

Outcome

One language layer can describe equations, workflows, engine calls, and analysis contracts without changing representation.

Near-Term Build

1. Define the symbolic core and canonical node types.
2. Add rewrite rules and validation passes.
3. Bind syntax objects to Nexus pipelines and managed environments.
4. Evaluate data products through the same object graph used for execution.
5. Emit reproducible manifests for execution and analysis.

goal: a compact language substrate for reproducible computational science inside Calyr

Calyr_ai_syntax

symbolic clarity before execution complexity

CALYRAI × BOKU VIENNA

Prepared for selective institutional presentation use

Calyrai Project

Three Core Computational Engines

AlphaFold

Input: sequence, PAE, confidence
Output: initial monomer geometry
Role: structural prior for SBPA

LAMMPS

Input: unit cell, restraints, force field
Output: constrained MD trajectory
Role: dynamic physical refinement

OpenMM

Input: topology, initial coords
Output: GPU-accelerated trajectory
Role: flexible force field MD on cluster

All three engines are Nexus-registered tools. No engine runs outside the Nexus control plane.

Engines as Nexus-Bound Tools

A computational engine that is not declared in Nexus does not exist in Calyrai.

- Each engine is registered in `nexus/core/registry.yaml`
- Invocation is always through a declared pipeline step
- The engine changes; the declaration language does not

Registry

```
valid_tools:  ["alphafold", "lammps", "openmm", "vsc_submit"]
```

VSC-5: Remote Execution Surface

Cluster Parameters

Host: vsc5.vsc.ac.at
Scheduler: SLURM
Max runtime: 72 hours
User: trupert
Project: BOKU CF BINF (#71863)
Home: /home/fs71863/trupert
Data: /gpfs/data/fs71863/trupert

Declaration unchanged

The same Nexus pipeline that runs locally runs on VSC-5. Only the executor field changes.

Execution surface = *where*.
Workflow declaration = engine-agnostic.

Installation Pipeline on VSC

Pattern: check → install → verify

```
# Full installation pipeline
nexus run vsc_install_computational_engines

# Stage-scoped
nexus run vsc_install_computational_engines \
  --stage check_availability
nexus run vsc_install_computational_engines \
  --stage verify_installation
```

All three stages write traceable output to `nexus/runs/`

FAIR for Remote Runs

- **Findable:** every VSC job has a `run_id` derived from pipeline name + timestamp
- **Accessible:** output paths declared in `nexus/core/vsc_config.yaml` and stable
- **Interoperable:** environment spec is a versioned conda YAML in Git
- **Reusable:** same declaration reruns identically on any compatible cluster

The W remains constant. The machine changes.

Nexus Data Layer

Declarative query

```
{entity:  scattering_curves,
 filter:  {quality:  high}}
```

↓ *translated to:*

Resolved SQL

```
SELECT * FROM scattering_curves
WHERE quality_score >= 0.8;
```

FAIR query log

query_hash: `a55bef0...`
timestamp: `2026-03-26T...`
reproducible: `true`

Same query ⇒ same hash.
Data access is as auditable as code.

From Simulation to Construction

Forward (simulation)

$$y = F(x)$$

Given a state x, predict outcome y.

Inverse (construction)

$$\text{given } y^* : \text{ find } x \text{ such that } F(x) \approx y^*$$

Choose interventions to produce a target state.

Construction is control, not only prediction.

The Qualitative Jump

Level	Core question
Description	What is?
Simulation	What happens if...?
Construction	What must I do so that...?

- Many classical fitting systems are simulatable but not constructable.
- Degenerate fits and non-identifiability block stable inversion.
- Calyrai addresses this with constraints + closed-loop execution.

Nexus as Construction Machine

Target $\rightarrow$ constrained inversion $\rightarrow$ candidates $\rightarrow$ experiment $\rightarrow$ feedback $\rightarrow$ update

- **Representations:** real space, Fourier space, energy landscape
- **Constraints:** stability, geometry, interfaces, packing
- **Loop:** propose, test, learn, iterate

Inference $\rightarrow$ Control $\rightarrow$ Design is the operational transition.

Limits and Honest Boundaries

- Not every system is fully constructable.
- Limits: high dimensionality, incomplete data, nonlinear couplings, non-identifiability.
- Therefore: iterative approximation, probabilistic candidates, robust correction layers.

Simulatability enables prediction. Constructability enables control. Closed loops make both operational.